



Responsive Web Design (Part 1)

Assoc. Prof. Dr. Kanda Runapongsa Saikaew
Faculty of Engineering
Khon Kaen University





Agenda



- **What is Responsive Web Design?**
- Viewport
- Grids
- Media Queries



What is Responsive Web Design?



- Responsive web design makes your web page look good on all devices
- Responsive web design uses only HTML and CSS
- Responsive web design is not a program or a JavaScript



Design for the Best, Experience for All Users



- Web pages can be viewed using many different devices
 - Desktops
 - Tablets
 - Phones
- Your web page should look good, and be easy to use, regardless of the device



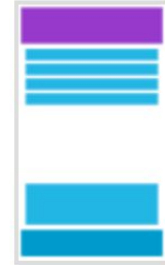
Web Pages should Fit any Device



Desktop



Tablet



Phone



Web Page on a Desktop



Chania

The Flight

The City

The Island

The Food

The City

Chania is the capital of the Chania region on the island of Crete. The city can be divided in two parts, the old town and the modern city.

What?

Chania is a city on the island of Crete.

Where?

Crete is a Greek island in the Mediterranean Sea.

How?

You can reach Chania airport from all over Europe.

Resize the browser window to see how the content respond to the resizing.



Web Page on a Mobile



The Flight

The City

The Island

The Food

The City

Chania is the capital of the Chania region on the island of Crete. The city can be divided in two parts, the old town and the modern city.

What?

Chania is a city on the island of Crete.

Where?

Crete is a Greek island in the Mediterranean Sea.

How?

You can reach Chania airport from all over Europe.

Resize the browser window to see how the content respond to the resizing.



Agenda



- What is Responsive Web Design?
- **Viewport**
- Grids
- Media Queries



What is the Viewport?



- A viewport is an area that is visible to the user on the browser window
- A bigger screen will have a larger viewport and a smaller screen will have a viewport less in number



Introduction to Viewport Meta Tag



- Crucial for responsive web design
- Controls how a webpage is displayed on mobile devices
- Instructs the browser on how to render the page's dimensions and scaling
- Without it, mobile browsers use a virtual viewport (typically 980px wide)



Important of Viewport Meta Tag



- Ensures proper rendering across various devices
- Prevents mobile browsers from using their default viewport size
- Allows designers to optimize content for smaller screens
- Improves user experience by eliminating the need for horizontal scrolling



Basic Viewport Meta Tag Syntax



- Placed in the `<head>` section of HTML document
- Basic syntax: `<meta name="viewport" content="width=device-width, initial-scale=1">`
- `width=device-width`: Sets viewport width to device's screen width
- `initial-scale=1`: Sets initial zoom level when page is first loaded



Viewport Settings for Responsive Design



- **width=device-width** part sets the width of the page to follow the screen-width of the device (which will vary depending on the device)
- **initial-scale=1.0** part sets the initial zoom level when the page is first loaded by the browser
 - Increasing the initial scale value to two will zoom to 2x



Impact on Mobile Rendering



- Proper viewport settings prevent horizontal scrolling
- Ensures text and images are appropriately sized for the device
- Allows CSS media queries to work correctly
- Improves readability and usability on small screens



Why max-width: 100% for responsive images:



- **Prevents pixelation**
 - Image never stretches beyond its natural dimensions, maintaining original quality
- **Smart scaling**
 - Scales down when container is smaller, stays original size when container is larger
- **Quality preservation**
 - width: 100% forces stretching that degrades image quality on large screens
- **Flexible behavior**
 - Adapts to content rather than forcing content to adapt to container
- **Industry standard**
 - Combined with height: auto, it's the universally recommended approach for responsive images



Best Practices about Viewport



- Always include a viewport meta tag in responsive designs
- Test your site on various devices and screen sizes
- Be cautious with fixed viewport widths or disabling user scaling
- Consider combining with responsive images and flexible layouts for best results



Without and with the Viewport



17:41



To understand this example, you should open this page on a phone or a tablet.



Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed diam nonummy nibb euismod tincidunt ut laoreet dolore magna aliquam erat volutpat. Ut wisi enim ad minim veniam, quis nostrud exerci tation ullamcorper suscipit lobortis nisl ut aliquip ex ea commodo consequat. Duis autem vel eum iriure dolor in hendrerit in vulputate velit esse molestie consequat, vel illum dolore eu feugiat nulla facilisis at vero eros et accumsan et justo odio dignissim qui blandit praesent luptatum zril deneit augue dui dolor te feugiat nulla facilisis. Nam liber tempor cum soluta nobis eleifend option congue nihil imperdiet doming id quod mazim placerat facer possim assum. Nam liber tempor cum soluta nobis eleifend option congue nihil imperdiet doming id quod mazim placerat facer possim assum.

<> without_viewport.html •

responsive > <> without_viewport.html > html > head > meta

```
1 <!DOCTYPE html>
2 <html>
3   <head>
4     <meta http-equiv="Content-Type" content="text/html; />
5   </head>
6   <body>
```

AA krunapon.github.io



17:40



To understand this example, you should open this page on a phone or a tablet.



Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed diam nonummy nibb euismod tincidunt ut laoreet dolore m erat volutpat. Ut wisi enim ad minim veniam, xerci tation ullamcorper suscipit lobortis nisl ut commodo consequat. Duis autem vel eum iriure rerit in vulputate velit esse molestie consequat, ire eu feugiat nulla facilisis at vero eros et uto odio dignissim qui blandit praesent l delenit augue dui dolor te feugait nulla liber tempor cum soluta nobis eleifend option mperdiet doming id quod mazim placerat facer i. Nam liber tempor cum soluta nobis eleifend : nihil imperdiet doming id quod mazim placerat assum.

AA krunapon.github.io



<> with_viewport.html M X

responsive > <> with_viewport.html > html > head > meta

You, 4 seconds ago | 2 authors (You and one other)

```
1 <!DOCTYPE html>
2 <html>
3   <head>
4     <meta http-equiv="Content-Type" content="text/html; />
5     <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6   <style>
7     img {
8       max-width: 100%;
9       height: auto;
10    }
11  </style>
12 </head>
13 <body>
```



Size Content to the Viewport



- Do not use large fixed width elements
- Do not let the content rely on a particular viewport width to render well
- Use CSS media queries to apply different styling for small and large screens



Agenda



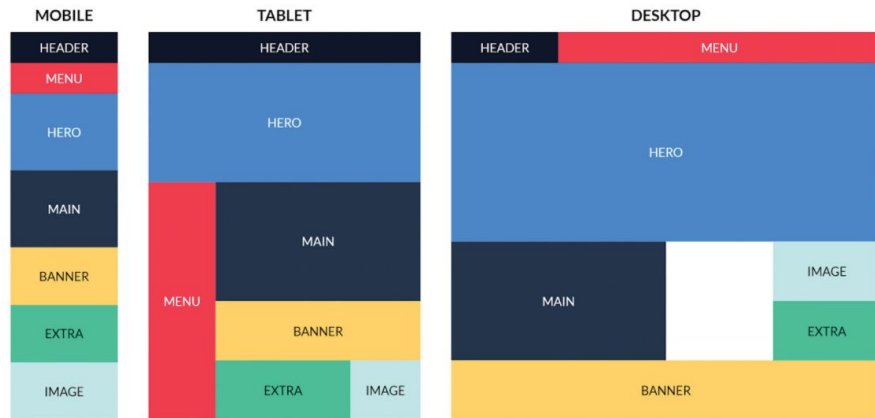
- What is Responsive Web Design?
- Viewport
- **Grids**
- Media Queries



CSS Grids



- CSS Grids are used to create content divided into multiple rows or columns





Grids are responsive in nature



- The grid boxes will automatically change their sizes, i.e, width and height when the webpage is loaded onto a smaller screen
- To make an element in the “grid” structure, you need to specify the grid property in the display attribute

```
1 class_name {  
2  
3 display: grid  
4  
5 }
```



CSS Grid for Responsive Layouts



- CSS Grid: Two-dimensional layout system
- Powerful tool for creating complex, responsive layouts
- Key features: fr units, minmax(), auto-fit/auto-fill
- Enable flexible and dynamic grid structures



Understanding fr Units



- fr = “fraction of available space”
- Distributes space proportionally among grid items
- Example: grid-template-columns: 1fr 2fr 1fr;
- Create three columns: 25%, 50%, 25% of available width



CSS Grids fr Units Demonstration



```
.container {  
  display: grid;  
  grid-template-columns: 1fr 2fr 1fr;  
  gap: 20px;  
  max-width: 1000px;  
  margin: 0 auto;  
}  
  
.item {  
  background-color: #3498db;  
  color: white;  
  padding: 20px;  
  text-align: center;  
  border-radius: 5px;  
}
```

```
<h1>CSS Grid fr Units Demonstration</h1>  
  
<div class="container">  
  <div class="item">1fr</div>  
  <div class="item">2fr</div>  
  <div class="item">1fr</div>  
</div>
```

CSS Grid fr Units Demonstration

1fr

2fr

1fr



Introducing minmax()



- `minmax(min,max)`: Sets a size range for grid tracks
- Ensures columns/rows are never smaller than min or larger than max
- Example: `minmax(100px, 1fr)`
- Useful for creating responsive layouts without media queries



CSS Grid minmax() Function Demonstration



minmax(100px, 1fr)

minmax(200px, 2fr)

Try resizing your browser window to see how the grid responds!

Explanation

This example demonstrates the use of the minmax() function in CSS Grid:

```
.container {  
  display: grid;  
  grid-template-columns: minmax(100px, 1fr) minmax(200px, 2fr);  
  gap: 20px;  
}
```

- The grid has two columns.
- The first column:
 - Minimum width: 100px
 - Maximum width: 1fr (1 fraction of available space)
- The second column:
 - Minimum width: 200px
 - Maximum width: 2fr (2 fractions of available space)
- There is a 20px gap between the columns.

minmax(100px, 1fr)

minmax(200px, 2fr)

Try resizing your browser window to see how the grid responds!

Explanation

This example demonstrates the use of the minmax() function in CSS Grid:

```
.container {  
  display: grid;  
  grid-template-columns: minmax(100px, 1fr) minmax(200px, 2fr);  
  gap: 20px;  
}
```

- The grid has two columns.
- The first column:
 - Minimum width: 100px
 - Maximum width: 1fr (1 fraction of available space)
- The second column:
 - Minimum width: 200px
 - Maximum width: 2fr (2 fractions of available space)
- There is a 20px gap between the columns.

The minmax() function sets a size range for grid tracks. When the container is wide, the columns will use the fr units to distribute space. When the container is narrow, they will respect their minimum widths.



Auto-fit and auto-fill



- Used with repeat() to create flexible number of grid items
- Auto-fit: Fits as many items as possible in the row
- Auto-fill: Creates as many tracks as will fit, even if empty
- Enables responsive layouts without media queries



Auto-fit vs. auto-fill

auto-fit

```
.container {  
  display: grid;  
  grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));  
  gap: 20px;  
}
```

auto-fit will expand items to fill the entire row. If there are not enough items to fill the row, the grid will adjust the size of the items to fit the available space.

auto-fit



auto-fill



Try resizing your browser window and toggling the number of items to see how auto-fit and auto-fill behave differently!

auto-fit



auto-fill



Try resizing your browser window and toggling the number of items to see how auto-fit and auto-fill behave differently!

auto-fill

```
.container {  
  display: grid;  
  grid-template-columns: repeat(auto-fill, minmax(200px, 1fr));  
  gap: 20px;  
}
```

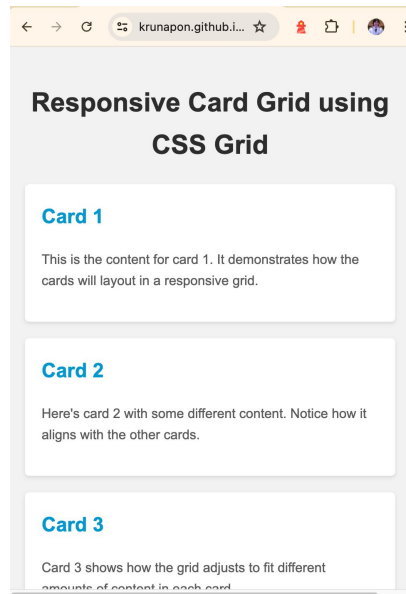
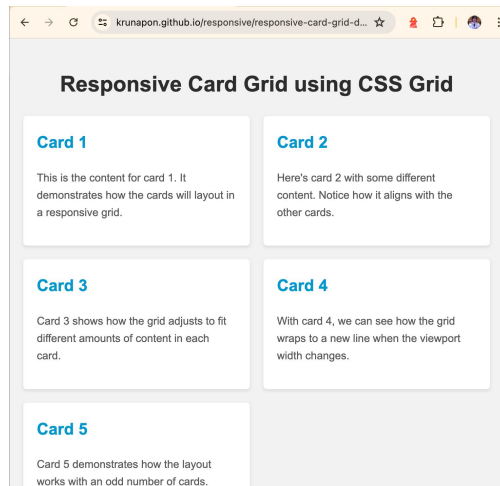
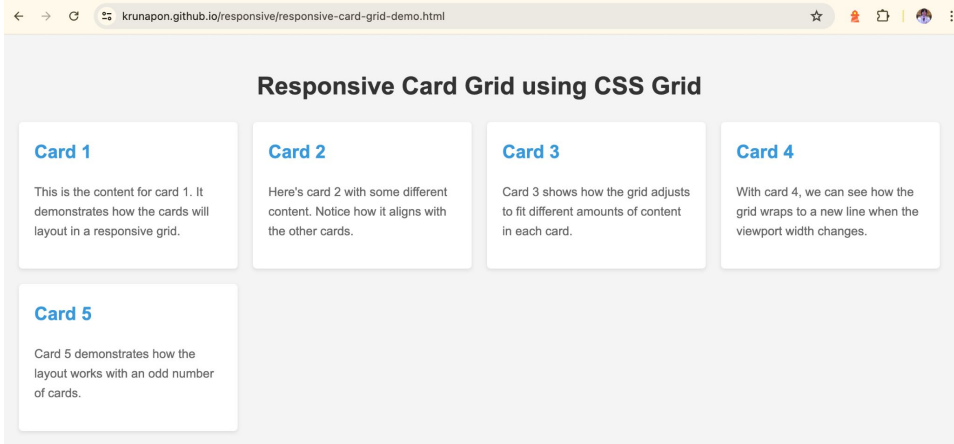
auto-fill will create as many columns as can fit based on the specified minimum width (200px in this case), even if there are not enough items to fill these columns. This can result in empty tracks.



Responsive Card Grid Using CSS Grid



<https://krunapon.github.io/responsive/responsive-card-grid-demo.html>





CSS Grid Explanation



```
.container {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
  gap: 20px;
  max-width: 1200px;
  margin: 0 auto;
}

.card {
  background-color: #fff;
  border-radius: 5px;
  box-shadow: 0 2px 5px rgba(0,0,0,0.1);
  padding: 20px;
}

.card h2 {
  color: #3498db;
  margin-top: 0;
}

.card p {
  color: #666;
}
```

```
<div class="container">
  <div class="card">
    <h2>Card 1</h2>
    <p>This is the content for</p>
  </div>
  <div class="card">
    <h2>Card 2</h2>
    <p>Here's card 2 with some</p>
  </div>
  <div class="card">
    <h2>Card 3</h2>
    <p>Card 3 shows how the gr</p>
  </div>
</div>
```

- Display: grid; Establishes a grid container
- Grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
 - repeat(auto-fit,...) - Creates as many columns as will fit in the container
 - minmax(250px, 1fr) - Each column will be at least 250px wide, but can grow larger if space allows
- Equal heights are free: grid items stretch to the tallest item in their row by default (align-items: stretch)
- gap: 20px; Creates a 20px gap between grid items both horizontally and vertically



Centering Text



- Centering text horizontally
 - `text-align: center`
- Centering text vertically
 - Using padding
 - `padding: <top> <right> <bottom> <left>`
 - `padding: <top-bottom> <right-left>`
 - `padding: 50px 0`



Building a Responsive Grid-View



- First ensure that all HTML elements have the box-sizing property set to border-box
- This makes the padding and border are included in the total width and height of the elements

```
* {  
  box-sizing: border-box;  
}
```



Without and with box-sizing



← → ↻ 🔒 krunapon.github.io/grid-view1-no.html

Covid-19

- The Origin
- The Stats
- The Vaccine

Covid-19 News

With a new wave of Covid-19 reported in Myanmar, the Interior Ministry has ordered tightened security along the border to prevent illegal immigration, and strict health checks on people arriving through official crossings.

Resize the browser window to see how the content respond to the resizing.

← → ↻ localhost:8000/grid-view1.html

Covid-19

- The Origin
- The Stats
- The Vaccine

Covid-19 News

With a new wave of Covid-19 reported in Myanmar, the Interior Ministry has ordered tightened security along the border to prevent illegal immigration, and strict health checks on people arriving through official crossings.

Resize the browser window to see how the content respond to the resizing.



Without and with box sizing



```
<style>
.header {
  border: 1px solid red;
  padding: 15px;
}

.menu {
  width: 25%;
  float: left;
  padding: 15px;
  border: 1px solid red;
}

.main {
  width: 75%;
  float: left;
  padding: 15px;
  border: 1px solid red;
}
</style>
```

```
<style>
* {
  box-sizing: border-box;
}

.header {
  border: 1px solid red;
  padding: 15px;
}

.menu {
  width: 25%;
  float: left;
  padding: 15px;
  border: 1px solid red;
}

.main {
  width: 75%;
  float: left;
  padding: 15px;
  border: 1px solid red;
}
</style>
```



Agenda



- What is Responsive Web Design?
- Viewport
- Grids
- **Media Queries**



Media Queries



- Media queries allow responsive design based on device characteristics
- Can target screen size, resolution, orientation, and more
- Complex queries combine multiple conditions
- Essential for creating adaptive layouts across devices



Basic Media Query Syntax



- General syntax: `@media [media-type] [media-feature]`
- Media types: screen, print, speech, all
- Common features: width, height, orientation
- Example: `@media screen and (max-width: 600px) {...}`

```
@media media type and (condition: breakpoint) {  
  // CSS rules  
}
```

```
@media only screen and (max-width: 600px) {  
  body {  
    background-color: lightblue;  
  }  
}
```



Media Types



- If we don't apply a media type, the @media rule selects all types of devices by default
- Otherwise, Media types come right after the @media rule
- There are many kinds of devices but we can group them into 4 categories
 - all - for all media types
 - print - for printers
 - screen - for computer screens, tablets, and smart-phones
 - speech - for screen readers that “read” the page out loud



Parenthesis



- Inside the parentheses, we set a condition
- I want to apply a larger font size for mobile devices
- To do that, we need to set a maximum width which checks the width of a device
- Important: **always put your media queries at the end of your CSS file**

```
.text {  
  font-size: 14px;  
}  
  
@media (max-width: 480px) {  
  .text {  
    font-size: 16px;  
  }  
}
```



Combining Multiple Conditions



- Use 'and' to combine conditions
- Use command for 'or' logic
- Nesting for complex combinations
- Can mix different types of media features
- Example: tablet in landscape mode

@media (min-width: 768px) and (max-width: 1024px)
and (orientation: landscape) { ... }



Complex Media Query Examples



- High-res display on larger screens
 - `@media (min-width: 1200px) and (min-resolution: 192dpi) { ... }`
- Target devices with mouse and large screens
 - `@media (pointer: fine) and (min-width: 1200px) { ... }`
- Styles for print or color printers
 - `@media print and (color) { ... }`



Breakpoints



- Breakpoints are maybe the most common term you will hear and use
- A breakpoint is a key to determine when to change the layout and adapt the new rules inside the media queries
- Here, the breakpoint is 480px
 - Basically, if the width of a device is smaller than 480px, the text class will be applied, otherwise, it won't

```
@media (max-width: 480px) {  
  .text {  
    font-size: 16px;  
  }  
}
```



Orientation in CSS Media Query



- The orientation of a device is one of the major problems that break the responsiveness of your website

```
1 @media (orientation: landscape){  
2     p {  
3         color: red;  
4         font-size: 36px;  
5     }  
6 }
```



Orientation: Portrait/Landscape



- Media queries can also be used to change layout of a page depending on the orientation of the browser.
- You can have a set of CSS properties that will only apply when the browser window is wider than its height, a so called "landscape" orientation

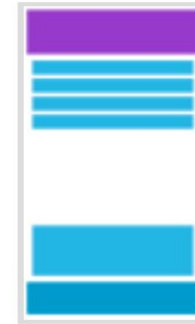
```
@media only screen and (orientation: landscape) {  
  body {  
    background-color: lightblue;  
  }  
}
```



Example of Adding a Breakpoint



Desktop



Phone



Another Breakpoint



- You can add as many breakpoints as you like.
- We will also insert a breakpoint between tablets and mobile phones
- We do this by adding one more media query (at 600px), and a set of new classes for devices larger than 600px (but smaller than 768px)



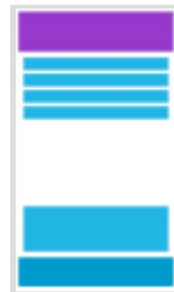
Desktop vs. Tablet vs. Phone



Desktop



Tablet



Phone



Common breakpoints



- 320px — 480px: Mobile devices
- 481px — 768px: iPads, Tablets
- 769px — 1024px: Small screens, laptops
- 1025px — 1200px: Desktops, large screens
- 1201px and more — Extra large screens, TV



Hiding an Element in One Size Range Only (mobile-first)



Base style hides it; a **min-width** query brings it back

```
.tagline {  
  display: none;  
} /* hidden by default */
```

```
@media (min-width: 481px) {  
  .tagline { display: block; } /* visible ≥481px */  
}
```



Mobile-First vs Desktop-First Approaches



- Two main strategies for responsive design
- Affects how media queries are written
- Impacts the default styles of your website
- Choose based on project requirements and target audience



Mobile-First Approach



- Start with styles for smallest screens
- Use min-width media queries to add complexity for larger screens
- Prioritizes mobile experience
- Often results in lighter base CSS



Mobile-First Example



`/* Base styles for mobile */`

```
.container { width: 100%; }
```

`/* Tablet styles */`

```
@media (min-width: 768px) {
```

```
  .container { width: 750px; }
```

```
}
```

`/* Desktop styles */`

```
@media (min-width: 1200px) {
```

```
  .container { width: 1170px; }
```

```
}
```



Desktop-First Approach



- Start with styles for largest screens
- Use max-width media queries to simplify for smaller screens
- Prioritize desktop experience
- Can be quicker to prototype complex layouts



Desktop-First Example



`/* Base styles for desktop */`

```
.container { width: 1170px; }
```

`/* Tablet styles */`

```
@media (max-width: 1199px) {
```

```
  .container { width: 750px; }
```

```
}
```

`/* Mobile styles */`

```
@media (max-width: 767px)
```

```
{
```

```
  .container { width: 100%; }
```

```
}
```



Choosing Between Mobile-First and Desktop-First



- Consider your target audience and their primary devices
- Think about the complexity of your design
- Mobile-first often leads to more efficient CSS
- Desktop-first can be easier for complex, desktop-heavy sites



Pick One Strategy and Stay Consistent



- mobile-first uses only **min-width**,
- desktop-first uses only **max-width**
- Mixing them in the same layout is error-prone



Advanced Mobile-First Example



```
/* Mobile base styles */
```

```
body { font-size: 16px; }
```

```
.nav { display: none; }
```

```
/* Tablet styles */
```

```
@media (min-width: 768px) {
```

```
  body { font-size: 18px; }
```

```
  .nav { display: block; }
```

```
}
```

```
/* Desktop styles */
```

```
@media (min-width: 1200px) and  
(min-resolution: 192dpi) {
```

```
  body { font-size: 20px; }
```

```
  .hi-res-image { display: block; }  
}
```



Advanced Desktop-First Example



`/* Desktop base styles */`

```
.sidebar { width: 300px; float: right; }  
.main-content { margin-right: 320px; }
```

`/* Tablet styles */`

```
@media (max-width: 1024px) {  
  .sidebar { width: 200px; }  
  .main-content { margin-right: 220px; }  
}
```

`/* Mobile styles */`

```
@media (max-width: 767px) {  
  .sidebar, .main-content {  
    width: 100%;  
    float: none;  
    margin-right: 0;  
  }  
}
```



CSS Units Absolute vs Relative



- Absolute unit: **px** - fixed size; ignores user and parent settings
- Relative units scale from a reference point
 - **em** - relative to the font size of the current/parent element
 - **rem** - relative to the root (<html>) font-size
- Browsers default the root font-size to 16px -> **1 rem = 16 px**
- Relative units let type and spacing scale together



Why rem Matters for Responsive Design



- Change one value (html font-size) -> every rem-based size scales together
- Respect the user's browser font-size setting -> better **accessibility** (users who enlarge default text)
- Avoid em's compounding in nested elements (1.5em inside 1.5em = 2.25x)
- Recommended for font-size, spacing (padding/margin), and media-query breakpoints



rem in Practice



```
html { font-size: 16px; } /* 1rem = 16px */
```

```
h1 { font-size: 2rem; } /* 32px */
```

```
p { font-size: 1rem; } /* actually don't need to specify */
```

```
.card { padding: 1.5rem; } /* 24px */
```

```
/* Scale the whole page by changing ONE value */
```

```
@media (min-width: 1200px) {
```

```
  html { font-size: 18px; } /* now 1rem = 18px → everything scales up */
}
```



CSS float Property



- The float property is used to specify how an element should be positioned horizontally within its container
- Floated elements are taken out of the normal flow of the document and shift to the left or right side of their containing elements



CSS float Property Example



```
.demo {
  max-width: 640px;
  border: 2px solid #333;
  padding: 15px;
  /* Modern Clearfix: forces this
  container to fully contain its
  floated children */
  display: flow-root;
}

.box {
  width: 110px;
  height: 110px;
  background: #1f9bd1;
  color: #fff;
  font-weight: bold;
  display: flex;
  align-items: center;
  justify-content: center;
}
```

← → ↻ ⓘ 127.0.0.1:5500/css/float-demo.html ☆ 👤

CSS Float Property Demonstration

Float Left

This is some text to demonstrate how floated elements interact with other content. Notice how each line is shortened to fit the narrow channel that remains between the two floated boxes: the left box sits against the left edge, the right box against the right edge, and the text flows down the middle. The lines stay narrow line after line until the text passes the bottom of the taller floated box. After that point the lines reclaim the full width of the container and resume their normal flow below the floats, exactly as they would if no floats were present at all.

Float Right

Explanation

This example demonstrates the CSS `float` property:

- The left blue box has `float: left;`
- The right blue box has `float: right;`

Notice how:

- The two boxes are pulled out of the block flow and pinned to the left and right edges of the container.
- The paragraph stays in normal flow, and its lines wrap into the channel between the boxes.
- Once the text clears the bottom of the boxes, it returns to full width.
- The container has `display: flow-root;` (a modern clearfix) so it grows to contain the floated boxes instead of collapsing.



CSS float Property Example



```
.float-left { float: left; margin: 0 15px 15px 0; }
.float-right { float: right; margin: 0 0 15px 15px; }
/* Remove the paragraph's top margin so its first line
starts level with the
boxes — makes the wrapping beside the boxes immediately
visible. */
.demo p { margin-top: 0; }
</style>
</head>
<body>
<h2>CSS Float Property Demonstration</h2>
<div class="demo">
<!-- The floated boxes MUST appear BEFORE the paragraph in
the source order,
otherwise the following text cannot wrap around them.
-->
<div class="box float-left">Float Left</div>
<div class="box float-right">Float Right</div>
<p>This is some text ...</p>
```

← → ↺ 127.0.0.1:5500/css/float-demo.html ☆

CSS Float Property Demonstration

Float Left

This is some text to demonstrate how floated elements interact with other content. Notice how each line is shortened to fit the narrow channel that remains between the two floated boxes: the left box sits against the left edge, the right box against the right edge, and the text flows down the middle. The lines stay narrow line after line until the text passes the bottom of the

Float Right

taller floated box. After that point the lines reclaim the full width of the container and resume their normal flow below the floats, exactly as they would if no floats were present at all.

Explanation

This example demonstrates the CSS float property:

- The left blue box has float: left;
- The right blue box has float: right;

Notice how:

- The two boxes are pulled out of the block flow and pinned to the left and right edges of the container.
- The paragraph stays in normal flow, and its lines wrap into the channel between the boxes.
- Once the text clears the bottom of the boxes, it returns to full width.
- The container has display: flow-root; (a modern clearfix) so it grows to contain the floated boxes instead of collapsing.



Best Practices for Media Queries



- Group media queries at the end of your CSS file
- Use variables or preprocessors for breakpoints
- Test on real devices, not just emulators
- Consider using em or rem units for breakpoints
- Combine different techniques for optimal results
- Always consider user experience across all target devices



Practice Exercise (1/5)



In this exercise, you'll create a responsive navigation bar that adapts to different screen sizes using viewport settings and CSS Grid.

1. Create a navigation bar with 5 items: Home, About, Services, Portfolio, and Contact.
2. Use CSS Grid to layout the navigation items.
3. For screens wider than 768px:
 - Display all items in a single row
 - Use equal width for all items
4. For screens 768px and narrower:
 - Display items in a single column
 - Each item should take up the full width of the screen
5. Ensure the webpage is responsive on both desktop and mobile devices using appropriate viewport settings.



Practice Exercise (2/5)



```
// Starting HTML
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Responsive Navigation Bar</title>
  <!-- Add your viewport meta tag here -->
  <style> /* Add your CSS here */ </style>
</head>
```

```
<body>
  <nav>
    <a href="#home">Home</a>
    <a href="#about">About</a>
    <a href="#services">Services</a>
    <a href="#portfolio">Portfolio</a>
    <a href="#contact">Contact</a>
  </nav>
</body>
</html>
```



Practice Exercise (3/5)



Tasks:

1. Add the appropriate viewport meta tag to ensure proper rendering on mobile devices.
2. Style the navigation bar using CSS Grid.
3. Implement the responsive behavior using a media query.
4. Test your solution by resizing the browser window or using browser developer tools to simulate different device sizes.



Practice Exercise (4/5)

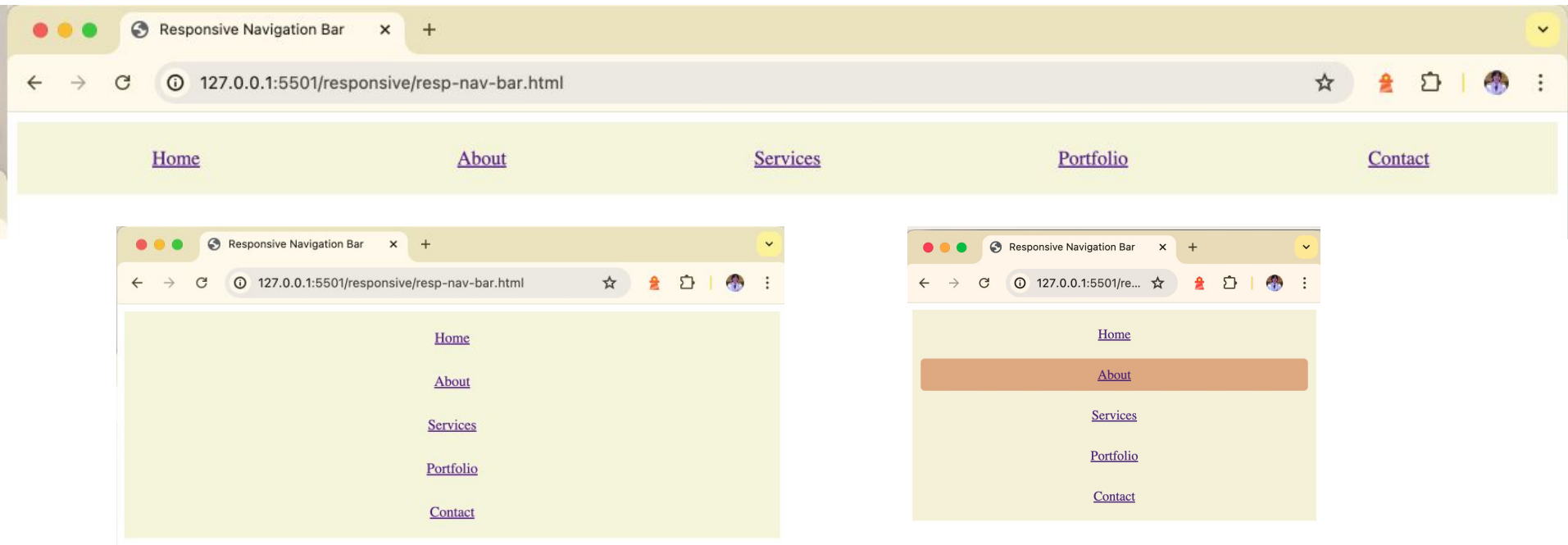


Hints:

- Use `display: grid` on the `nav` element.
- Consider using `grid-template-columns: repeat(5, 1fr)` for the desktop layout.
- For the mobile layout, you might use `grid-template-columns: 1fr`.
- Don't forget to add some padding and styling to make the navigation items look like buttons.



Practice Exercise (5/5)





Practice Exercise Solution



```
<title>Responsive Navigation Bar</title>
```

```
<meta name="viewport"
```

```
content="width=device-width, initial-scale=1.0" />
```

```
<style>
```

```
nav {
```

```
    display: grid;
```

```
    background-color: beige;
```

```
    padding: 10px;
```

```
    grid-template-columns: repeat(5, 1fr);
```

```
    gap: 10px;
```

```
    margin: 0 auto;
```

```
}
```

```
nav a {
```

```
    border-radius: 5px;
```

```
    text-align: center;
```

```
    padding: 10px;
```

```
}
```

```
nav a:hover {
```

```
    background-color: burlywood;
```

```
}
```

```
@media (max-width: 768px) {
```

```
    nav {
```

```
        grid-template-columns: 1fr;
```

```
    }}
```



References



- https://www.w3schools.com/css/css_rwd_intro.asp
- <https://blog.hubspot.com/website/w3-css-vs-bootstrap>
- <https://css-tricks.com/all-about-floats/>
- <https://www.lambdatest.com/blog/css-media-queries-for-responsive-design/>
- <https://www.lambdatest.com/blog/responsive-web-design-all-you-need-to-know/>
- [https://www.lambdatest.com/blog/how-to-make-responsive-images-with-css-html-wordpre ss-more/](https://www.lambdatest.com/blog/how-to-make-responsive-images-with-css-html-wordpress-more/)
- <https://www.webfx.com/blog/web-design/html5-frameworks/>
- <https://www.lambdatest.com/blog/how-to-use-css-breakpoints-for-responsive-design/>
- https://www.w3schools.com/css/css3_flexbox_responsive.asp
- <https://www.olympic-museum.de/pictograms/olympic-games-pictograms-2021.php>
- <https://blog.logrocket.com/tailwind-css-vs-bootstrap-ui-kits/>
- <https://claude.ai>

